CitizenAI

AI-Powered Civic Issue Reporting & Resolution Platform

Complete Project Documentation & Implementation Blueprint

ARCHITECTURE: React.js → Spring Boot (Java) → Flask (Python AI Service) → AI Models | PostgreSQL/PostGIS → persistent application data

This document is a build-from-scratch blueprint. It makes concrete architecture decisions, defines responsibilities, APIs, database design, AI pipelines, testing and deployment rather than giving a generic technology list.

Version 1.0 | August 2026

Document Contents

- 1. Executive Summary
- 2. Problem, Objectives and Scope
- 3. Functional and Non-Functional Requirements
- 4. System Architecture and Responsibility Boundaries
- 5. Detailed Project/Folder Structure
- 6. Technology Stack and Decisions
- 7. AI/ML Models and Pipelines
- 8. Business Logic and Workflows
- 9. Database Design and Data Flow
- 10. API Structure
- 11. Spring Boot ↔ Flask Contract
- 12. Authentication, Authorization and Security
- 13. React Frontend Breakdown
- 14. Spring Boot Backend Breakdown
- 15. Flask AI-Service Breakdown
- 16. Hardware and Software Requirements
- 17. Testing Strategy
- 18. Docker and Deployment
- 19. Performance, Scalability, Logging and Monitoring
- 20. Risks and Mitigations
- 21. Development Roadmap
- 22. Final Checklist
- Appendices

1. Executive Summary

CitizenAI is an AI-assisted civic operations platform. A citizen submits an image, optional text/voice description and GPS location. The platform detects the civic problem, understands the complaint, checks for duplicates, estimates severity, identifies the responsible department, manages the work lifecycle, and assists with repair verification.

Core lifecycle: REPORT → UNDERSTAND → DETECT → DEDUPLICATE → PRIORITIZE → ROUTE → RESOLVE → VERIFY → LEARN → PREDICT.

Project positioning

- This is not merely a complaint CRUD application.
- It is not the first AI civic system. Existing research and deployments already cover detection, reporting and analytics.
- The proposed contribution is the integrated closed-loop workflow plus a strict React/Spring Boot/Flask separation.
- Human officers remain responsible for consequential decisions; AI produces evidence and recommendations.

Initial civic categories

- Pothole
- Road crack/surface damage
- Garbage accumulation
- Illegal dumping
- Broken streetlight
- Drainage overflow
- Water leakage
- Road obstruction

Primary users

User	Responsibilities
Citizen	Submit issues, view status, see nearby public issues, provide resolution feedback.
Municipal Officer	Validate AI evidence, assign work, update status, submit repair evidence.
Administrator	Manage users, departments, wards, categories, configuration, analytics and audit.

2. Problem, Objectives and Scope

Problem statement

Civic reporting is often fragmented and manual. Citizens may select the wrong category, provide incomplete context or create duplicate complaints. Authorities spend time classifying, prioritizing and routing reports. Closure may be administrative rather than evidence-based, and historical complaint data is underused for prevention.

Objectives

- Provide image, text and voice reporting with automatic GPS capture.
- Detect supported civic issues with a custom-trained YOLO model.
- Structure multilingual complaints using speech recognition and an LLM where useful.
- Cluster probable duplicate reports into one physical Civic Issue.
- Generate an explainable severity/priority score.
- Route issues using GIS, issue category, ward and department rules.
- Provide officer queues, SLA tracking, status history and auditability.
- Assist before/after repair verification and citizen confirmation.
- Provide GIS heatmaps and operational analytics.
- Create a foundation for predictive maintenance.

Scope

In scope	Not in first release
React citizen/officer/admin UI	Autonomous government action
Spring Boot APIs/business logic/auth/DB	Direct traffic-signal or infrastructure control
Flask AI service	Nationwide production deployment
PostgreSQL/PostGIS + object storage	Legal enforcement workflows
Detection, duplicates, severity, routing, verification	Large-scale prediction before enough data exists

3. Functional and Non-Functional Requirements

Functional requirements

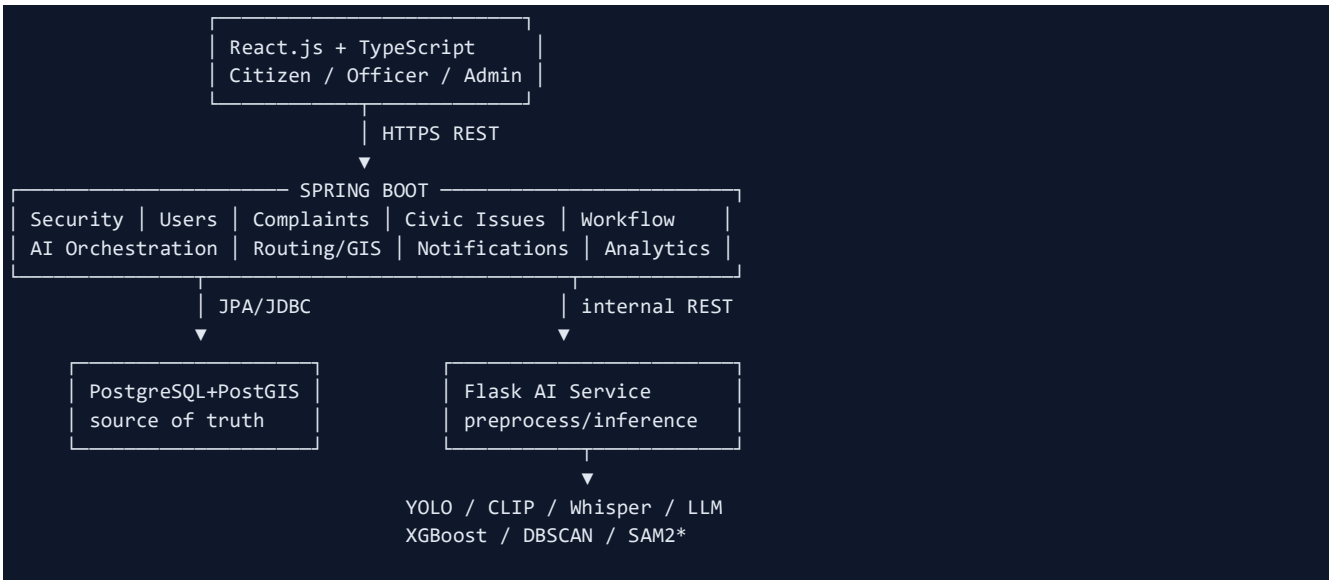
ID	Area	Requirement
FR-01	Authentication	Registration, login, refresh, logout, password reset, role-aware access.
FR-02	Complaint creation	Image/video, text/voice, GPS, timestamp and optional category.
FR-03	AI analysis	Create AI job, call Flask, validate and persist result.
FR-04	Duplicate detection	Find candidates and link reports to a shared Civic Issue.
FR-05	Severity	Calculate score/priority and store explanation factors.
FR-06	Routing	Resolve ward, department, SLA and work queue.
FR-07	Workflow	Controlled status transitions with authorization.
FR-08	Officer operations	Review, assign, progress, repair evidence and escalation.
FR-09	Citizen tracking	Timeline, notifications and resolution feedback.
FR-10	Analytics	Ward/category/department trends, hotspots and SLA metrics.
FR-11	Administration	Manage users, roles, departments, wards and categories.
FR-12	Audit	Record critical security, workflow and AI override actions.

Non-functional requirements

- Security: HTTPS, strong password hashing, JWT validation, RBAC, validation, protected media and internal service authentication.
- Reliability: complaint creation must not depend on Flask availability; AI jobs are retryable.
- Performance: normal application endpoints target sub-second response under prototype load; AI inference is measured separately.
- Scalability: stateless Spring Boot, independently scalable AI workers, indexed PostGIS and object storage.
- Maintainability: DTOs, service boundaries, migrations, tests, API versioning and model versioning.
- Observability: correlation IDs, structured logs, metrics, health/readiness checks.
- Privacy: minimize PII, protect GPS/media, define retention and deletion rules.

4. System Architecture and Responsibility Boundaries

Overall architecture



Responsibility rules

Component	Owns	Must NOT own
React	UI, forms, client validation, maps, API calls, presentation state	DB access, business rules, AI inference, secrets
Spring Boot	Auth, RBAC, APIs, business logic, transactions, DB, workflow, routing, AI orchestration, audit	Python ML dependencies/model execution
Flask	AI preprocessing, inference, postprocessing, AI schemas and model health	Users, complaints, workflow, business authorization
Database	Persistent application state, relationships, GIS data	UI or model inference
Object storage	Media/model artifacts	Complaint workflow state

Browser → Spring Boot only. Spring Boot → Flask internally. Flask never becomes a second application backend.

5. Detailed Hierarchical Project / Folder Structure

```
citizenai/
├── frontend/
│   ├── src/
│   │   ├── app/                    # router/providers/auth state
│   │   ├── components/             # reusable UI
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── complaints/
│   │   │   ├── civic-issues/
│   │   │   ├── map/
│   │   │   ├── notifications/
│   │   │   ├── dashboard/
│   │   │   └── admin/
│   │   ├── services/               # Spring API clients only
│   │   ├── hooks/
│   │   ├── types/
│   │   └── utils/
│   ├── public/  package.json  Dockerfile
│
├── backend/
│   ├── src/main/java/com/citizenai/
│   │   ├── config/ security/ auth/ user/
│   │   ├── complaint/ civicissue/ workflow/
│   │   ├── ai/ routing/ gis/ notification/
│   │   ├── analytics/ admin/ audit/
│   │   ├── common/ exception/
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── db/migration/
│   ├── pom.xml
│   └── Dockerfile
│
├── ai-service/
│   ├── app/
│   │   ├── api/ schemas/ preprocessing/
│   │   ├── inference/ postprocessing/
│   │   ├── models/
│   │   │   ├── detection/ embeddings/ speech/
│   │   │   ├── llm/ severity/
│   │   ├── services/ config/ main.py
│   ├── model_weights/ tests/
│   ├── requirements.txt  Dockerfile
│
├── database/migrations/ seed/
├── datasets/raw/ processed/ metadata/
├── infra/nginx/ monitoring/ compose/
├── docs/architecture/ api/ research/
├── docker-compose.yml
├── .env.example
└── README.md
```

- Never commit model weights, raw private uploads, secrets or huge datasets to Git.
- Use object storage/model artifact storage and keep checksums/version metadata.

6. Technology Stack and Architectural Decisions

Technology	Layer	Why
React 19 + TypeScript 5.x	Frontend	Typed, component-based UI and strong ecosystem for dashboards/maps.
Spring Boot 3.5.x + Java 21 LTS	Main backend	Strong transactions, security ecosystem, maintainable business-layer architecture.
Spring Web + Validation	API	Typed REST controllers and server-side validation.
Spring Security	Security	JWT/resource-server patterns, RBAC and endpoint protection.
Spring Data JPA/Hibernate	Persistence	Repository pattern, ORM and transactions.
Flyway	Migrations	Versioned database schema changes.
PostgreSQL 16 + PostGIS 3.x	DB/GIS	Relational integrity plus spatial search and ward polygons.
Flask 3.x + Python 3.11	AI service	Lightweight HTTP boundary for Python ML ecosystem.
YOLOv8s	Vision	Good speed/accuracy tradeoff and straightforward custom training.
CLIP	Vision embeddings	Visual similarity for duplicate candidate detection.
Whisper	Speech	Local-capable voice-to-text.
Qwen-family multilingual LLM	NLP	Structured multilingual complaint understanding; local inference is possible.
XGBoost	ML	Strong tabular severity model.
DBSCAN	Clustering	Natural fit for spatial duplicate grouping.
Leaflet + OpenStreetMap	Maps	Flexible, low-cost prototype mapping.
Docker + Compose	Deployment	Reproducible multi-service development/deployment.
Nginx	Edge	Reverse proxy, TLS termination and routing.
JUnit/Mockito/Testcontainers	Java tests	Unit + integration coverage.
Pytest	AI tests	Preprocessing/model/API tests.
Playwright	E2E	Browser-level workflow validation.

Version note: exact patch versions should be pinned at implementation time. The versions above are recommended baselines; the lockfiles and build files are the final source of truth.

7. AI/ML Models, Inputs, Outputs and Pipelines

Model	Purpose	Input	Preprocess	Output	Postprocess
YOLOv8s	Civic detection	Image	Resize/letterbox, RGB normalization	Class + confidence + bbox	NMS + threshold + domain class mapping
CLIP	Visual similarity	Image/crop	Model resize/normalize	Embedding	Cosine similarity
Whisper	Speech-to-text	Audio	Resample/normalize	Transcript + language	Text cleanup
Qwen-family LLM	Complaint structuring	Text/transcript	Constrained prompt + category vocabulary	Structured JSON	Schema validation
XGBoost	Severity	Structured features	Missing-value handling/encoding	Score/probability	Policy maps score → priority
DBSCAN	Duplicate grouping	Location + similarity features	Normalize/weight features	Cluster labels	Link reports to Civic Issue
SAM2 (optional)	Damage segmentation	Image + detector region	Crop/prompt	Pixel mask	Area estimation / verification

YOLO training strategy

- Use pretrained YOLOv8s weights; do not train from scratch.
- Build a unified CitizenAI taxonomy from public datasets plus custom Indian civic images.
- Start with five core classes, then expand to eight or ten after reliable evaluation.
- Use train/validation/test splits with a final untouched test set.
- Report mAP@50, mAP@50-95, precision, recall and per-class confusion.

AI pipeline

```
Image → Flask preprocessing → YOLOv8s → NMS/threshold → structured detections
     → Spring validates result → severity/routing business logic → DB

Voice → Whisper → transcript → Qwen structured extraction → Spring → DB

Duplicate → PostGIS candidate search → CLIP/text similarity → DBSCAN/rules
          → existing Civic Issue OR new Civic Issue

Repair → before/after evidence → optional segmentation/similarity
       → verification recommendation → officer/citizen confirmation
```

AI confidence is not complaint priority. YOLO confidence answers 'how likely is this object?' Severity answers 'how urgently should the civic issue be handled?'

8. Business Logic and Major Workflows

Complaint creation

1. Citizen submits media + optional text/voice + GPS.
2. Spring Boot validates identity, media limits and request data.
3. Complaint is persisted in SUBMITTED state before AI processing.
4. Media goes to object storage; metadata goes to PostgreSQL.
5. AI job is created and sent to Flask; failure does not delete the complaint.
6. Spring Boot validates the AI response and applies domain rules.

Duplicate detection

```
New complaint
   ↓
PostGIS candidates within radius
   ↓
CLIP image similarity + text similarity + time proximity
   ↓
weighted duplicate score
   ├─ HIGH → link to existing Civic Issue
   ├─ MEDIUM → officer review
   └─ LOW → create new Civic Issue
```

Severity

Features can include issue category, estimated affected area, nearby school/hospital, road type, traffic/footfall if available, report count, historical issue density, weather context and repeat failures. XGBoost produces a calibrated score; Spring Boot maps the score to LOW/MEDIUM/HIGH/CRITICAL using explicit policy thresholds.

Routing

```
GPS → ward polygon → issue category → department mapping → SLA policy → priority queue → officer/team
```

Resolution

```
ASSIGNED → IN_PROGRESS → REPAIR_SUBMITTED → AI_VERIFICATION → RESOLVED / HUMAN_REVIEW / REOPENED
```

Important rule

AI may recommend. Spring Boot state transitions and authorized humans decide. Flask must never directly mark a complaint as officially resolved.

9. Database Design, ER Model and Data Flow

ER model

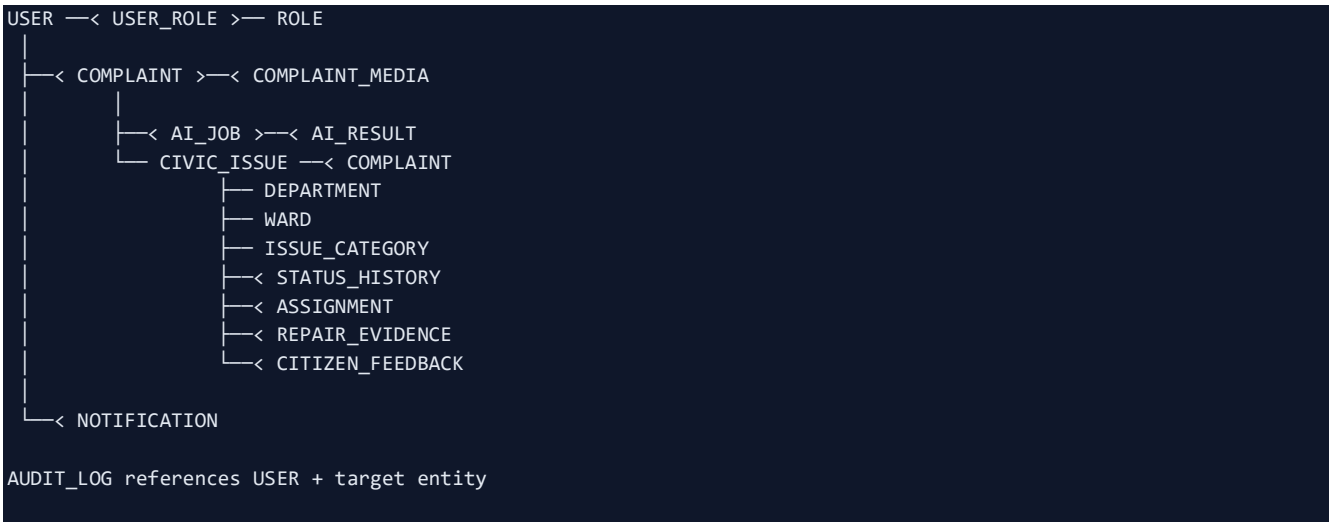


Table	Important columns/purpose
users	id, name, email, phone, password_hash, status, created_at
roles	id, code, name
user_roles	user_id, role_id
departments	id, code, name, active
wards	id, name, boundary_geometry, active
issue_categories	id, code, name, description, active
complaints	id, reporter_id, civic_issue_id, category_id, description, source, latitude, longitude, location_geom, status, priority, created_at
complaint_media	id, complaint_id, object_key, mime_type, size_bytes, checksum, created_at
civic_issues	id, category_id, ward_id, department_id, centroid_geom, severity_score, priority, status, created_at, updated_at
ai_jobs	id, complaint_id, job_type, status, model_version, request_id, started_at, completed_at, error_code
ai_results	id, ai_job_id, result_json, confidence, processing_ms, warnings
assignments	id, civic_issue_id, officer_id, assigned_at, due_at, completed_at
status_history	id, civic_issue_id, from_status, to_status, changed_by, reason, created_at
repair_evidence	id, civic_issue_id, media_id, verification_status, verification_score, reviewed_by, created_at
citizen_feedback	id, civic_issue_id, user_id, outcome, comment, created_at
notifications	id, user_id, type, payload_json, read_at, created_at
audit_logs	id, actor_user_id, action, entity_type, entity_id, metadata_json, created_at

Critical indexes

- users(email) UNIQUE
- complaints(status, priority, created_at)
- civic_issues(ward_id, status)
- GIST index on complaints.location_geom
- GIST index on civic_issues.centroid_geom
- ai_jobs(status, created_at)
- status_history(civic_issue_id, created_at)
- audit_logs(entity_type, entity_id, created_at)

Media binaries are stored in object storage; PostgreSQL stores ownership, checksum, type and object key. JSONB can store raw AI output while important fields such as category, priority and status remain normalized for reporting.

10. Complete Spring Boot API Structure

Method	Endpoint	Role	Purpose
POST	/api/v1/auth/register	Public	Create citizen account
POST	/api/v1/auth/login	Public	Login and tokens
POST	/api/v1/auth/refresh	Authenticated	Refresh access
GET	/api/v1/me	Authenticated	Current profile
POST	/api/v1/complaints	CITIZEN	Create complaint
GET	/api/v1/complaints/{id}	Scoped	Complaint details
GET	/api/v1/complaints	Scoped	Search/paginate complaints
POST	/api/v1/complaints/{id}/analyze	CITIZEN/OFFICER	Trigger/retry AI
GET	/api/v1/issues/{id}	Scoped	Civic issue
GET	/api/v1/issues/nearby	Authenticated	Nearby public issues
POST	/api/v1/issues/{id}/assign	OFFICER/ADMIN	Assignment
POST	/api/v1/issues/{id}/status	OFFICER/ADMIN	State transition
POST	/api/v1/issues/{id}/repair-evidence	OFFICER	Repair evidence
POST	/api/v1/issues/{id}/feedback	CITIZEN	Resolution feedback
GET	/api/v1/dashboard/summary	OFFICER/ADMIN	KPIs
GET	/api/v1/dashboard/heatmap	OFFICER/ADMIN	GIS heatmap
GET	/api/v1/admin/users	ADMIN	User management
PATCH	/api/v1/admin/users/{id}	ADMIN	User status/roles
GET	/api/v1/admin/audit	ADMIN	Audit search
GET	/api/v1/health	Internal/Public	Health/readiness

API rules

- Version base path from day one: /api/v1.
- Use JSON for structured data and multipart/form-data for media.
- Paginate all collections.
- Return 201 for creation, 202 for accepted async AI jobs, 409 for state conflicts and 422 for semantic validation.
- Never expose stack traces, SQL errors, model paths or secrets.
- Use idempotency keys for expensive/retriable creation or AI job requests where needed.

Example error envelope

```
{
  "timestamp": "2026-08-17T10:20:30Z",
  "status": 422,
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "fieldErrors": {"media": "Unsupported media type"},
  "correlationId": "c7b3..."
}
```

11. Spring Boot ↔ Flask AI Service Contract

Method	Endpoint	Request	Response
POST	/internal/v1/image/analyze	multipart image	detections + model version + timing
POST	/internal/v1/image/embed	image	embedding or embedding reference
POST	/internal/v1/speech/transcribe	audio	transcript + language
POST	/internal/v1/text/structure	text JSON	category + summary + context
POST	/internal/v1/severity/predict	feature JSON	score + suggested priority
POST	/internal/v1/repair/verify	before/after refs	verification score/status
GET	/internal/v1/health	none	service/model readiness

Example image result

```
{
  "request_id": "a4f7...",
  "model": "citizenai-yolov8s",
  "model_version": "1.0.0",
  "processing_ms": 184,
  "detections": [
    {"class_code":"POTHOLE","confidence":0.964,
     "bbox":{"x":0.42,"y":0.55,"w":0.31,"h":0.22}}
  ],
  "warnings":[]
}
```

Service communication rules

- Flask is private/internal and not browser-accessible.
- Use a service token or mTLS in production.
- Spring Boot sends and Flask returns a correlation/request ID.
- Spring validates every Flask response against a strict schema.
- On timeout, mark AI job RETRYABLE_FAILED and preserve the complaint.
- Malformed AI output is INVALID_RESULT; it must not mutate business state.

12. Authentication, Authorization, Security and Error Handling

Authentication flow

```
React → POST /api/v1/auth/login → Spring Security
     → verify password hash
     → short-lived access token + refresh mechanism
     → React calls protected Spring endpoints
```

Authorization

Role	Capabilities
CITIZEN	Own complaints, public nearby issues, feedback, notifications.
OFFICER	Department/ward-scoped review, assignment, workflow, repair evidence.
ADMIN	Global configuration, users, analytics, audit.
SYSTEM	Internal service identity for jobs/schedulers.

Security controls

- BCrypt or Argon2 password hashing.
- TLS/HTTPS externally; private network for Flask and database.
- Strict upload size, MIME and content validation; generate server-side object keys.
- JPA/parameterized queries; never build SQL from raw user input.
- Rate-limit login, complaint creation and expensive AI endpoints.
- Object-level authorization: citizens cannot access other users' private complaints.
- Audit admin actions, status overrides and AI overrides.
- Minimize PII sent to Flask; protect GPS and media.
- Secrets via deployment secret management, never Git.

Error handling

Use Spring's global exception handler to convert validation, authentication, authorization, not-found, state-conflict and downstream-AI errors into stable API error codes. Flask should use a parallel structured error schema. Correlation IDs must appear in client responses and logs.

13. React Frontend Module Breakdown

Module	Responsibilities	Screens/components
Auth	Session, protected routes, login/register	Login, Register, Profile, RouteGuard
Citizen reporting	Capture/upload media, GPS, description, submit	NewComplaint, MediaUploader, LocationPicker
Complaint tracking	Evidence, timeline, feedback	ComplaintDetails, StatusTimeline
Nearby issues	Map and filters	MapView, IssueMarker, Filters
Officer dashboard	Priority queue, assignment, SLA	WorkQueue, IssuePanel, AssignmentForm
Admin	Users, departments, wards, categories	UserAdmin, Config
Analytics	KPIs, charts, heatmaps	Dashboard, Heatmap, TrendChart
Notifications	Status/assignment updates	NotificationCenter

Frontend implementation rules

- All network calls use typed service modules pointing only to Spring Boot.
- Use TanStack Query or equivalent for server-state caching and request lifecycle.
- Use client-side validation for UX; server validation remains authoritative.
- Do not compute final severity, routing or authorization in React.
- Display AI confidence and review state rather than pretending AI is always correct.
- Map only privacy-appropriate public data to citizens.

14. Spring Boot Backend Module Breakdown

Package	Responsibility
security	JWT, filters, RBAC, password hashing, CORS/CSRF policy.
auth	Registration/login/refresh/account state.
user	Profiles, roles and user lifecycle.
complaint	Creation, media metadata, ownership, search.
civicissue	Issue aggregate and linked reports.
workflow	State machine, transitions, status history.
ai	Flask client, job orchestration, retries, result validation.
routing	Ward lookup, category→department mapping, SLA policy.
gis	PostGIS queries, nearby issues and aggregates.
notification	In-app/email/push abstraction.
analytics	KPI/trend/department/ward queries.
admin	Configuration and administration.
audit	Critical action logs.
common/exception	DTOs, correlation IDs, error handling.

Layering

```
controller → HTTP only
dto        → API contracts
service    → business logic + transactions
entity     → persistence/domain model
repository → data access
client     → Flask/internal service clients
mapper     → DTO/domain mapping
config/security → infrastructure/cross-cutting
exception  → global error handling
```

Put business rules in services, not controllers. Put persistence in repositories. Keep controllers thin.

15. Flask AI-Service Breakdown

Responsibilities

- Load/cache model artifacts at startup.
- Validate and normalize images/audio.
- Run model preprocessing and inference.
- Apply NMS, confidence thresholds and domain class mapping.
- Return strict Pydantic response schemas.
- Expose model health/readiness and version metadata.
- Keep model inference isolated from application database and user management.

Structure

```
ai-service/app/
├── api/              # HTTP routes
├── schemas/          # Pydantic request/response contracts
├── preprocessing/    # image/audio normalization
├── inference/        # model calls
├── postprocessing/   # thresholds, NMS, mapping
├── models/           # wrappers for YOLO/CLIP/Whisper/LLM/XGB
├── services/         # orchestration inside AI boundary
├── config/
└── main.py
```

Flask must not

- Authenticate end users.
- Read/write complaint tables directly.
- Own complaint lifecycle or status.
- Assign departments through business rules.
- Store user profiles.

Model lifecycle

Training happens offline and is versioned. Deployment loads a specific model artifact. Every inference result stores model name/version. Retraining creates a new artifact version and should not silently replace a production model.

16. Hardware and Software Requirements

Resource	Minimum development	Preferred AI development
CPU	4 cores x86-64	8+ cores
RAM	16 GB	32 GB+
GPU	Optional	NVIDIA 8–16 GB+ VRAM preferred for training/LLM experiments
Storage	50 GB free	200 GB+ SSD
OS	Windows 11 or Ubuntu 22.04+	Ubuntu 22.04/24.04 preferred
Network	Internet for setup/datasets	Stable high-speed connection

Software

- JDK 21 LTS
- Node.js LTS + npm/pnpm
- Python 3.11
- PostgreSQL 16 + PostGIS
- Git
- Docker Desktop / Docker Engine + Compose
- IntelliJ IDEA or VS Code
- CUDA/PyTorch-compatible NVIDIA driver stack if using GPU

Offline-capable AI

The AI service can run locally using downloaded model weights, avoiding runtime external AI APIs. If fully offline mapping is later required, use locally hosted map tiles. Initial dataset/package acquisition still requires network access unless mirrored.

17. Testing Strategy

Layer	Tools	Coverage
React	Vitest/Jest + Testing Library	Forms, validation, state, permissions, components.
E2E	Playwright	Login → complaint → officer → resolution end-to-end.
Spring unit	JUnit 5 + Mockito	Services, state transitions, routing and authorization.
Spring integration	Spring Boot Test + Testcontainers	PostgreSQL/PostGIS, transactions, Flask client.
Flask	Pytest	Preprocessing, schemas, endpoints, error paths.
Model	Python evaluation scripts	mAP, precision, recall, F1, confusion matrix, per-class errors.
Security	OWASP-oriented tests	Broken access control, upload abuse, injection, auth/session.
Performance	k6/Gatling/JMeter	Concurrency, latency, DB and AI throughput.

Acceptance scenarios

- Complaint persists even if Flask is unavailable.
- Citizen cannot access another citizen's private complaint.
- Officer can operate only within authorized scope.
- Duplicate reports are linked without deleting original submissions.
- AI cannot directly mark an issue RESOLVED.
- Invalid media is rejected before inference.
- Illegal state transitions return 409.
- Critical transitions create status-history and audit records.
- Spatial queries use indexes and return correct nearby issues.
- Every AI result records model version and processing time.

18. Docker, Containerization and Deployment

Local architecture



Compose services

Service	Purpose
nginx	Reverse proxy, TLS, routing
frontend	Build/static React application
backend	Spring Boot API
ai-service	Flask model inference
postgres	PostgreSQL + PostGIS
minio (optional)	S3-compatible object storage
redis (optional)	Queue/cache for async jobs

Production architecture

- Public edge only exposes Nginx/load balancer.
- Spring Boot and Flask scale independently.
- PostgreSQL should preferably be managed with backups and point-in-time recovery.
- Media goes to S3-compatible object storage.
- Long AI jobs use a queue/worker pattern.
- AI workers can run on GPU nodes while application nodes remain CPU-based.

19. Performance, Scalability, Logging and Monitoring

Targets and measurements

Area	Initial target/design
Spring API reads	p95 < 500 ms under expected prototype load, excluding AI.
Writes	p95 < 1 s for metadata operations; uploads depend on file size/network.
AI	Benchmark p50/p95 per model on actual hardware; do not invent a fixed latency.
DB	Indexes + pagination + EXPLAIN for spatial/aggregate queries.
AI concurrency	Bounded workers to avoid GPU/CPU exhaustion.
Media	Size limits, compression and object storage.

Scalability

- Keep Spring Boot stateless and horizontally scalable.
- Use asynchronous AI jobs and retry queues.
- Scale Flask GPU workers independently.
- Use PostGIS spatial indexes and pre-aggregated dashboard queries.
- Cache public map aggregates and static configuration.
- Move notifications to a queue as scale increases.

Structured logging

```
Request fields:
correlation_id, user_id, route, method, status, latency_ms

AI fields:
request_id, job_id, model, model_version,
processing_ms, status, confidence_summary, error_code

NEVER log:
passwords, JWTs, private media, raw audio, unnecessary PII
```

Monitoring

- Spring Boot Actuator health/readiness.
- Prometheus + Grafana for API latency, DB pool, queue depth and inference timing.
- Centralized JSON logs.
- Alerts on high 5xx rate, Flask failure rate, queue backlog, DB saturation and inference degradation.

20. Risks, Constraints and Mitigations

Risk	Problem	Mitigation
Insufficient labeled data	Poor local accuracy	Public datasets + custom Indian images; targeted collection; hard test set.
Class imbalance	Weak rare-class performance	Sampling/augmentation; class-aware evaluation.
False positives	Wrong issue/priority	Confidence thresholds + human review.
False duplicate merges	Different issues linked	Combine image + GPS + text + time; medium confidence review.
Government integration unavailable	Cannot test real authority system	Mock departments/wards; define adapter interface for future integration.
Flask outage	AI unavailable	Persist complaint first; retry AI job.
Large media	Storage/cost/latency	Size limits, compression, object storage, lifecycle policy.
Privacy	PII/location exposure	RBAC, minimization, encryption and audit.
Model drift	Accuracy falls	Monitor feedback; scheduled retraining/versioning.
LLM hallucination	Bad structured data	Constrained schema, fixed categories, validation, human review.
Scope explosion	Project cannot finish	Freeze MVP before advanced prediction/IoT/CCTV.

21. Development Phases and Implementation Roadmap

Phase	Focus	Deliverable
0	Architecture	Requirements, repo, Docker, schema, API conventions.
1	Backend core	Spring Boot, PostgreSQL/PostGIS, auth, users, departments, wards, complaints.
2	React MVP	Citizen report, login, tracking, officer dashboard, map.
3	AI detection	Dataset, annotation, YOLOv8s training, Flask endpoint.
4	AI intelligence	Whisper, text structuring, duplicate detection, severity.
5	Workflow	Routing, assignment, SLA, notifications, audit.
6	Verification	Repair evidence, before/after analysis, citizen confirmation.
7	Analytics	Heatmaps, trends, ward/department metrics.
8	Hardening	Security, performance, observability, deployment.
9	Final evaluation	Unseen test set, metrics, demo, research comparison.

Recommended 16-week plan

- Weeks 1–2: requirements, architecture, schema, Git/Docker.
- Weeks 3–5: Spring Boot core, DB, authentication and complaint workflow.
- Weeks 4–6: React citizen/officer UI.
- Weeks 6–9: dataset curation, YOLO training and Flask integration.
- Weeks 9–11: duplicate detection, multilingual NLP and severity.
- Weeks 11–13: routing, notifications, resolution verification and analytics.
- Weeks 14–15: testing, security, performance and deployment.
- Week 16: final evaluation, demo and documentation.

MVP freeze: React reporting UI + Spring Boot auth/database/workflow + Flask YOLO inference + five core classes + duplicate candidates + severity + routing + officer dashboard + status tracking.

22. Final Implementation Checklist

Architecture

- ☐ React calls only Spring Boot
- ☐ Spring Boot owns business logic and DB
- ☐ Flask is internal and inference-only
- ☐ AI contracts are versioned
- ☐ Model artifacts are outside Git

Backend

- ☐ Spring Security
- ☐ JWT/refresh strategy
- ☐ RBAC/object authorization
- ☐ Flyway migrations
- ☐ Global error handler
- ☐ Pagination/validation
- ☐ Workflow state machine
- ☐ Audit logging
- ☐ Flask timeout/retry

Database

- ☐ PostgreSQL/PostGIS
- ☐ Spatial indexes
- ☐ Foreign keys/constraints
- ☐ Seed wards/departments/categories
- ☐ AI job/result tables
- ☐ Backup plan

Frontend

- ☐ Protected routes
- ☐ Complaint form
- ☐ Media upload
- ☐ GPS handling
- ☐ AI processing state
- ☐ Officer queue
- ☐ Map/filter
- ☐ Citizen feedback
- ☐ Responsive layout

AI

- ☐ Dataset licenses
- ☐ Taxonomy
- ☐ Train/val/test
- ☐ YOLO baseline
- ☐ Per-class metrics
- ☐ Model versioning
- ☐ Whisper
- ☐ Duplicate pipeline

- ☐ Severity model
- ☐ Low-confidence review

Security

- ☐ HTTPS
- ☐ Password hashing
- ☐ Secret management
- ☐ Upload validation
- ☐ Rate limits
- ☐ Audit
- ☐ PII minimization
- ☐ Flask private

Testing

- ☐ Unit
- ☐ Integration
- ☐ AI endpoint
- ☐ Model evaluation
- ☐ E2E
- ☐ Authorization
- ☐ Failure/retry
- ☐ Performance

Deployment

- ☐ Dockerfiles
- ☐ Compose
- ☐ Nginx
- ☐ Health checks
- ☐ Object storage
- ☐ Monitoring
- ☐ Backups
- ☐ CI

Appendix A. Environment Variables

```
# Spring Boot
SPRING_PROFILES_ACTIVE=dev
DB_URL=jdbc:postgresql://postgres:5432/citizenai
DB_USERNAME=citizenai
DB_PASSWORD=change-me
JWT_SECRET=change-me
AI_SERVICE_BASE_URL=http://ai-service:8000
AI_SERVICE_TOKEN=change-me
OBJECT_STORAGE_ENDPOINT=http://minio:9000
OBJECT_STORAGE_BUCKET=citizenai-media
OBJECT_STORAGE_ACCESS_KEY=change-me
OBJECT_STORAGE_SECRET_KEY=change-me

# Flask
AI_SERVICE_TOKEN=change-me
MODEL_DIR=/models
YOLO_MODEL_PATH=/models/citizenai-yolov8s.pt
WHISPER_MODEL=...
LLM_MODEL_PATH=...
MAX_IMAGE_MB=10
MAX_AUDIO_MB=25

# React
VITE_API_BASE_URL=/api/v1
```

Production secrets must come from the deployment secret manager. The repository should contain only .env.example with placeholders.

Appendix B. Dataset and Research Notes

RDD2022 is a strong road-damage foundation: 47,420 images from six countries including India, with more than 55,000 annotated road-damage instances. TACO is useful for litter but its taxonomy does not exactly equal scene-level illegal dumping. Custom data means images curated and manually annotated specifically for the CitizenAI taxonomy; not every custom image must be photographed by the team.

The project should acknowledge existing civic-AI research, including recent work using YOLOv8s and Qwen2.5-7B-Instruct for civic issue reporting and work combining AI, GPS/GIS and notifications. The defensible contribution is the integrated closed-loop architecture and local-domain dataset, not the generic statement that 'AI is used for civic problems.'

Appendix C. Final Architecture Summary

```
React.js / TypeScript
        | HTTPS
        ▼
Spring Boot / Java 21
 ├─ Authentication + RBAC
 ├─ Users / Departments / Wards
 ├─ Complaints / Civic Issues
 ├─ Workflow + SLA + Routing
 ├─ Database operations
 ├─ Notifications + Analytics
 └─ AI orchestration
        | internal authenticated REST
        ▼
Flask / Python
 ├─ Preprocessing
 ├─ YOLOv8s
 ├─ CLIP
```

```
 ├─ Whisper
 ├─ Qwen-family LLM
 ├─ XGBoost
 └─ verification models
        │
        ▼
AI result → Spring validation → PostgreSQL
        │
        ▼
Officer action → Repair evidence
        │
        ▼
AI-assisted verification → Citizen confirmation
```

The core engineering rule: AI produces evidence and recommendations; Spring Boot owns application truth and authorized state transitions.

End of CitizenAI technical documentation.

23. Dataset Engineering & Model Training Specification

Purpose. This section defines exactly how the CitizenAI computer-vision dataset should be assembled, annotated, split, validated and versioned. The project must not treat 'download a dataset and train YOLO' as the complete ML workflow. Dataset quality will be one of the main determinants of model performance.

23.1 Dataset sources and role

Dataset / source	Primary use	CitizenAI classes / contribution	How it is used
RDD2022	Road damage detection	Pothole and road-crack related classes	Foundation dataset; map its labels into the CitizenAI taxonomy and retain only compatible labels.
TACO	Litter / waste imagery	Litter-related visual patterns	Useful foundation for waste detection; not sufficient by itself for scene-level illegal dumping.
Custom CitizenAI dataset	Local civic conditions	Garbage accumulation, illegal dumping, drainage overflow, water leakage, broken streetlight, road obstruction and additional local road examples	Core domain-adaptation dataset. Collect/curate and manually annotate according to the final taxonomy.
Custom hold-out test set	Final evaluation	All supported classes	Collected separately and never used for training or hyperparameter decisions.

Important: RDD2022 and TACO are supporting datasets, not the complete CitizenAI dataset. The custom Indian/local-condition dataset is essential because the target classes and visual conditions do not perfectly match public datasets.

23.2 Final v1 computer-vision taxonomy

ID	Class code	Class name	Description	Initial target images
0	POTHOLE	Pothole	Depressed/broken road surface forming a cavity.	800–1,200
1	ROAD_CRACK	Road crack	Longitudinal, transverse or surface cracking that is visibly relevant to road maintenance.	800–1,200
2	GARBAGE_ACCUMULATION	Garbage accumulation	Visible concentration of discarded waste in a public area.	800–1,200
3	ILLEGAL_DUMPING	Illegal dumping	Waste deliberately dumped in an unauthorized/open location.	800–1,200
4	BROKEN_STREETLIGHT	Broken streetlight	Streetlight visibly damaged, missing, broken or otherwise non-functional where visual evidence supports it.	600–1,000
5	DRAINAGE_OVERFLOW	Drainage overflow	Water/sewage visibly overflowing from a drain or drainage channel.	800–1,200
6	WATER_LEAKAGE	Water leakage	Visible leakage or uncontrolled water flow from a pipe, public fitting or infrastructure.	800–1,200
7	ROAD_OBSTRUCTION	Road obstruction	Object/material blocking or significantly obstructing a public road/path.	800–1,200

Target size: The table gives a practical starting target, not a scientific guarantee. Aim for roughly 6,000–9,000 usable images after cleaning if resources permit. A smaller dataset can still produce a prototype, but rare classes should not be reported as reliable merely because overall accuracy looks good.

23.3 Dataset directory structure

```
datasets/
├── raw/
│   ├── rdd2022/
│   ├── taco/
│   └── custom/
├── interim/
```

```
│   ├── cleaned/
│   ├── converted/
│   └── deduplicated/
├── annotations/
│   ├── yolo/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   └── review/
├── processed/
│   ├── train/
│   ├── val/
│   └── test/
├── metadata/
│   ├── sources.csv
│   ├── class_mapping.csv
│   ├── annotation_guidelines.md
│   ├── dataset_versions.json
│   └── test_manifest.csv
└── reports/
    ├── class_distribution.csv
    ├── quality_report.html
    └── evaluation/
```

23.4 Dataset acquisition and cleaning workflow

7. Record the source, license, download date and original label taxonomy for every public dataset.
8. Do not mix incompatible labels without an explicit mapping document.
9. Remove corrupted, unreadable and extremely low-information images.
10. Remove exact duplicates and near-duplicates before train/validation/test splitting.
11. Avoid placing frames from the same video or burst sequence across different splits because that causes data leakage.
12. Standardize image orientation and supported formats such as JPG/PNG.
13. Check class balance after conversion and again after the final split.
14. Create a separate manifest containing image ID, source, class, split, annotation status and dataset version.

23.5 Annotation policy

Recommended tool: CVAT or Label Studio for annotation, with YOLO-format export for training.

- Draw a tight bounding box around the visible target object/region.
- Do not include large irrelevant background areas merely to make annotation easier.
- If multiple civic issues of the same class appear, annotate each instance.
- If an issue is too ambiguous to identify confidently, mark it for review rather than forcing a label.
- Keep annotation rules consistent across public and custom images.
- For drainage overflow and water leakage, annotate the visually observable affected region, not an inferred underground source.
- For broken streetlights, only label visible evidence of damage/non-functionality; an unlit lamp in daytime is not enough evidence by itself.
- For illegal dumping versus ordinary garbage accumulation, maintain an explicit scene-level labeling guideline so the two classes do not become visually indistinguishable.

23.6 Train / validation / test split

Split	Recommended share	Purpose	Rule
Train	70%	Model fitting	May contain augmentation.
Validation	15%	Threshold/hyperparameter/model selection	Never use for final reported performance.
Test	15%	Final performance	Freeze before final evaluation; never tune on it.

The test set must be genuinely unseen. If 20 photographs of the same pothole are captured from nearly identical angles, do not distribute them across train and test. Group by physical event/location/source sequence first, then split.

23.7 Data augmentation

- Brightness/contrast variation for different lighting conditions.
- Small rotations and scale changes.
- Random crop/translation where the target remains visible.
- Moderate blur/noise to simulate mobile-camera conditions.
- Weather/illumination augmentation only when it remains realistic.
- Horizontal flipping only when it does not change the semantic meaning of the class.
- Do not use aggressive augmentation simply to inflate dataset size.

23.8 Dataset quality checks

Check	What to measure	Action if failed
Class balance	Images/instances per class	Collect targeted examples or use controlled sampling.
Annotation validity	Missing/invalid boxes, out-of-bounds coordinates	Fix or remove.
Duplicate leakage	Exact/near duplicate across splits	Regroup and resplit.
Image quality	Blur, darkness, unreadable content	Review; retain difficult cases only when realistic.
Label consistency	Same visual situation receiving different labels	Update guideline and re-annotate.
Source distribution	Public vs custom, city/region, weather	Ensure test set represents deployment conditions.
Small-object frequency	Very small streetlights/signs etc.	Collect closer/clearer examples and evaluate separately.

23.9 YOLO training configuration

```
Model: YOLOv8s pretrained weights
Input size: start at 640x640; benchmark 768/960 if small objects need it
Epochs: start around 80–150; use early stopping
Batch: largest stable batch for available VRAM
Optimizer: use Ultralytics defaults initially; tune only after baseline
Augmentation: realistic mobile-camera / weather variation
Validation: every training run
Final test: one frozen test set
Artifacts:
  - best.pt
  - last.pt
  - data.yaml
  - training config
  - class map
  - metrics.json
  - confusion_matrix.png
  - dataset_version
```

23.10 Model evaluation

- Report mAP@50 and mAP@50–95.
- Report precision, recall and F1 per class.
- Show a confusion matrix where meaningful.
- Measure false positives and false negatives for each civic class.
- Report performance separately on public-dataset images and custom/local images.
- Measure inference latency on the actual deployment hardware.
- Create a hard-case evaluation set containing night/low-light, rain, blur, occlusion, crowded scenes and unusual viewpoints.

23.11 Dataset versioning

Rule: Every deployed prediction must be traceable to a model version, and every model version must be traceable to a dataset version and training configuration.

23.12 Dataset collection plan for custom data

- Collect images from multiple locations rather than photographing one neighborhood repeatedly.
- Capture different distances, angles, times of day and weather conditions.
- Prefer real public infrastructure scenes over staged photographs.
- Do not intentionally capture identifiable faces, vehicle plates or private information when it is unnecessary.
- Obtain consent/permission where required for collection locations and contributors.
- Maintain a collection manifest: image ID, class candidate, approximate location/region, date, source and consent/license status.
- Have a second reviewer inspect a sample of annotations to estimate labeling consistency.

23.13 What is custom data?

Custom dataset means project-specific labeled data. It does not mean every image must be photographed by the team. Public licensed images can be included, but custom data is curated and annotated to fill the gaps between public datasets and the exact CitizenAI taxonomy, especially Indian/local visual conditions.

23.14 Recommended first training milestone

Do not start by training all eight classes simultaneously if annotation quality is uncertain. First build a five-class baseline: POTHOLE, ROAD_CRACK, GARBAGE_ACCUMULATION, DRAINAGE_OVERFLOW and ROAD_OBSTRUCTION. Prove the complete pipeline from image upload → Flask → YOLO → Spring Boot → database → UI. Then expand the taxonomy.

Appendix D. Dataset Deliverables Before Model Training

- ☐ Final class taxonomy and class_mapping.csv
- ☐ Dataset source/license manifest
- ☐ Annotation guideline document
- ☐ Cleaned and deduplicated image set
- ☐ YOLO-format labels
- ☐ Train/validation/test manifests
- ☐ Class-distribution report
- ☐ Annotation quality-review report
- ☐ data.yaml
- ☐ Baseline training configuration
- ☐ Frozen test-set manifest
- ☐ Model evaluation script
- ☐ Model card containing limitations and intended use